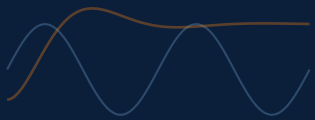
WEEK 2 · DAY 2 / 5

Mechatronics and Robotics Engineering (50 h)



Robot Mechanisms II and Introduction to Robotic Manufacturing

4.4 Inverse kinematics ▪ 4.5 Jacobian & statics ▪ 4.6 Dynamics ▪ 4.7 Motion & trajectories ▪ 5.1 Flexible automation

Mechatronics and Robotics Engineering Programme

Week 2 · Day 2 ▪ 2 weeks · 10 days · 50 hours

Agenda and learning outcomes

Code	Session	Time
4.4	Inverse kinematics: 2-link solution, multiple solutions, reach	1 h
4.5	Velocity kinematics: Jacobian, singularities, statics	1 h
4.6	Introduction to robot dynamics	1 h
4.7	Motion systems and trajectory planning	1 h
5.1	Robotic manufacturing and flexible automation	1 h
	Total (+ 10-min breaks about every hour)	**5 h**

One robot all day: a planar 2-link arm with $L_1 = 0.4\,\text{m}$, $L_2 = 0.3\,\text{m}$ (think of a SCARA seen from above, or a shoulder + elbow seen from the side).

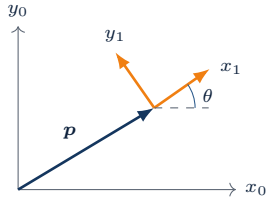
By the end of today you can . . .

- solve the 2-link inverse kinematics and find both solutions
- decide whether a point is reachable
- build the Jacobian and spot singularities
- calculate joint torques from a tip force ($\tau = J^{\mathsf{T}} F$)
- estimate gravity torques and explain payload effects
- compare PTP, linear and circular moves; compute an AMR's motion
- distinguish fixed, programmable and flexible automation

What we learned so far (W2D1)

- **3.7** Joints are springy: resonance, notch filters
- **4.1** Robot anatomy: joints, DOF, workspace; articulated, SCARA, gantry, delta, cobots, AMRs
- **4.2** Pose = position + orientation; rotation matrix R, homogeneous transform T
- **4.3** Forward kinematics: chain the transforms, joint angles $\rightarrow$ tool pose

> Today we turn the question around (where must the joints go?), then add speed, force and mass, and finally zoom out to the whole factory.



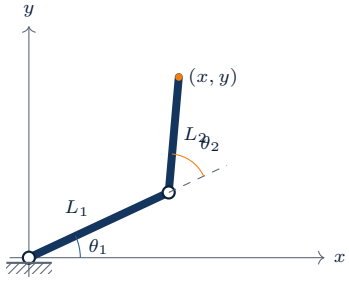
$$T = \begin{bmatrix} R & \boldsymbol{p} \\ \mathbf{0}^{\mathsf{T}} & 1 \end{bmatrix}$$

Recap: forward kinematics of our 2-link arm

- Joint angles θ_1 (shoulder), θ_2 (elbow, relative to link 1)
- Tip position = sum of two link vectors

$$x = L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2)$$
$$y = L_1 \sin\theta_1 + L_2 \sin(\theta_1 + \theta_2)$$

- Example: $\theta_1 = 0$, $\theta_2 = 90^\circ \Rightarrow x = 0.4\,\text{m}$, $y = 0.3\,\text{m}$
- Short-hand: $c_1 = \cos\theta_1$, $s_{12} = \sin(\theta_1 + \theta_2)$



Session 1

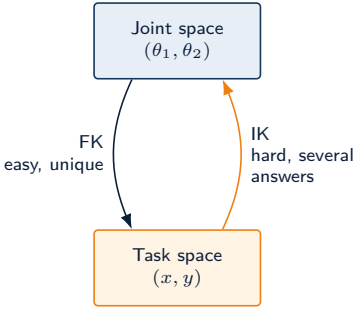
4.4 Inverse kinematics

The inverse question: where must the joints go?

- **Forward kinematics (FK)**: joints → tool pose. One answer, always.
- **Inverse kinematics (IK)**: desired tool pose → joint angles.
- IK is what we actually need: tasks are given in the world ("pick the part at x, y")
- Analogy: reaching for a cup – you think of the cup, your brain finds shoulder and elbow angles

Why IK is harder

Nonlinear (sin, cos), may have **0, 1, 2 or more** answers.

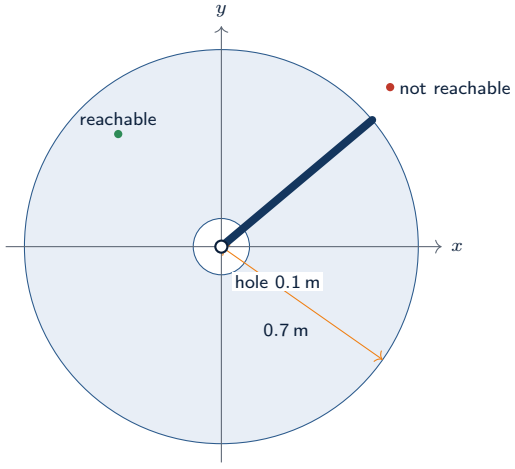


Reachability: the workspace of the 2-link arm

- Distance from base to target: $r = \sqrt{x^2 + y^2}$
- Fully stretched: $r_{\max} = L_1 + L_2 = 0.7\,\text{m}$
- Fully folded: $r_{\min} = |L_1 - L_2| = 0.1\,\text{m}$

$$|L_1 - L_2| \;\le\; \sqrt{x^2 + y^2} \;\le\; L_1 + L_2$$

- Outside the ring: **no solution** (unreachable)
- Real joint limits cut the ring further



Refresher: law of cosines and the two-argument arctangent

Refresher: law of cosines

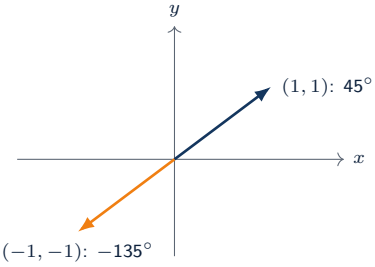
For a triangle with sides a, b, c and angle γ opposite c:

$$c^2 = a^2 + b^2 - 2ab\cos\gamma$$

Pythagoras is the special case $\gamma = 90°$.

Refresher: atan2

$\operatorname{atan2}(y, x)$ = angle of the point (x, y), in $(-180°, 180°]$. Unlike $\arctan(y/x)$ it knows the **quadrant**.



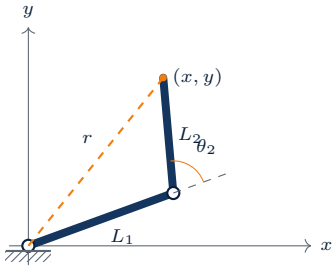
$\arctan(\frac{1}{1}) = \arctan(\frac{-1}{-1}) = 45°$: the plain arctangent cannot tell these two apart.

Geometric solution, step 1: the elbow angle

- Base, elbow and tip form a triangle with sides L_1, L_2, r
- Apply the law of cosines to this triangle (interior angle at the elbow $= 180^\circ - \theta_2$):

$$\cos\theta_2 = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}$$

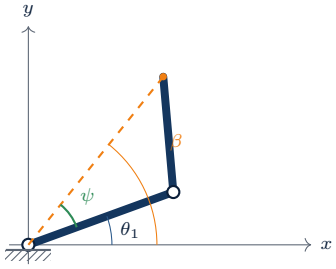
- If the right side is outside $[-1, 1]$: target **unreachable**
- $\theta_2 = \pm\arccos(\ldots) \rightarrow$ **two** elbow solutions



Geometric solution, step 2: the shoulder angle

- $\theta_1 =$ (direction to target) $-$ (angle between link 1 and the line to the target)

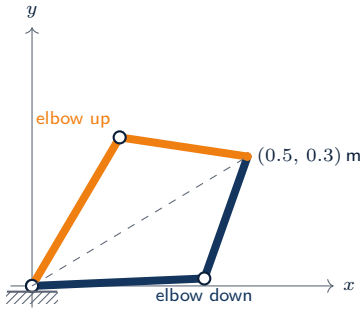
$$\theta_1 = \beta - \psi$$
$$\beta = \operatorname{atan2}(y,\, x)$$
$$\psi = \operatorname{atan2}\big(L_2 \sin\theta_2,\; L_1 + L_2 \cos\theta_2\big)$$



- Use the θ_2 from step 1 (each sign gives its own θ_1)
- Always use `atan2`, never $\arctan$ of a ratio
- Always check: put the answer back into FK

Two solutions: elbow up and elbow down

- Same tip position, two arm shapes (mirror images about the line base–target)
- $\theta_2 > 0$: one shape, $\theta_2 < 0$: the other
- On a SCARA these are called **right-handed** and **left-handed**
- The robot program must say which one – the **configuration**
- On the workspace boundary ($r = r_{\max}$) the two merge into one



Worked example: IK for the target $(0.5,\ 0.3)$ m

1. Reach: $r = \sqrt{0.25 + 0.09} = 0.583\,\text{m}$, inside $[0.1,\ 0.7]$ ✓
2. $\cos\theta_2 = \dfrac{0.34 - 0.16 - 0.09}{2 \cdot 0.4 \cdot 0.3} = \dfrac{0.09}{0.24} = 0.375$
   $\Rightarrow \theta_2 = \pm 68.0^\circ$
3. $\beta = \operatorname{atan2}(0.3, 0.5) = 31.0^\circ$;
   $\psi = \operatorname{atan2}(0.3 \cdot 0.927,\ 0.4 + 0.3 \cdot 0.375) =$
   $\operatorname{atan2}(0.278, 0.513) = 28.5^\circ$
4. Elbow down: $\theta_1 = 31.0 - 28.5 = 2.5^\circ$, $\theta_2 = +68.0^\circ$
   Elbow up: $\theta_1 = 31.0 + 28.5 = 59.5^\circ$, $\theta_2 = -68.0^\circ$

Check with FK (elbow down)

$$x = 0.4\cos 2.5^\circ + 0.3\cos 70.5^\circ = 0.400 + 0.100 = 0.50\,\text{m}\ \checkmark$$

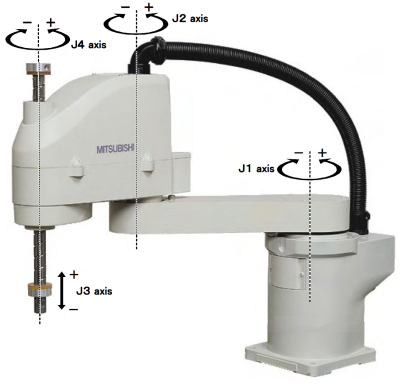
$$y = 0.4\sin 2.5^\circ + 0.3\sin 70.5^\circ = 0.017 + 0.283 = 0.30\,\text{m}\ \checkmark$$

For the elbow-up case ψ changes sign, because $\sin\theta_2 < 0$.

IK of real robots

- 6-axis arm: position *and* orientation → up to **8** solutions (shoulder front/back, elbow up/down, wrist flip/no flip)
- Most industrial arms have a **spherical wrist** (last 3 axes meet in a point) → IK splits into "arm finds the position, wrist finds the orientation"
- Joint limits remove some solutions
- Controller picks the one matching the programmed configuration, or the closest to the current pose

Going further: Robots without a closed-form solution use *numerical IK*: guess, compute the FK error, correct with the Jacobian (next session), repeat. Redundant robots (7 axes) have infinitely many solutions.

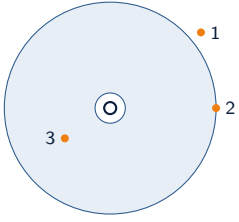


SCARA robot. Mitsubishi Electric Automation, CC BY-SA 4.0, via Wikimedia Commons

Check your understanding (4.4)

Check your understanding

1. Is the point $(0.6,\ 0.5)$ m reachable by our arm ($L_1 = 0.4$, $L_2 = 0.3$ m)? Why?
2. For the target $(0.7,\ 0)$ m, how many IK solutions are there? What are the joint angles?
3. A student computes θ_1 with $\arctan(y/x)$ for a target at $(-0.3,\ -0.2)$ m. What goes wrong?



Session 4.4 summary and key terms

- IK: from desired tool position to joint angles
- Reachable if $|L_1 - L_2| \le r \le L_1 + L_2$
- θ_2 from the law of cosines, θ_1 from two `atan2`s
- Two solutions (elbow up/down); a 6-axis arm has up to 8
- Always check by plugging back into FK

Key terms

inverse kinematics, **reachable workspace**, **atan2**, **configuration** (elbow up/down, lefty/righty), **spherical wrist**, **numerical IK**

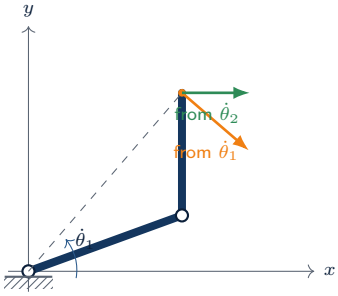
Session 2

4.5 Velocity kinematics: the Jacobian

From joint speeds to tip speed

- Turn the shoulder at $\dot{\theta}_1$: the whole arm swings, tip moves **perpendicular** to the line base–tip
- Turn the elbow at $\dot{\theta}_2$: only link 2 swings
- Tip speed = sum of the two contributions
- Analogy: a door – same rotation speed, the handle moves faster than a point near the hinge ($v = \omega r$)

How fast the tip moves depends on the *pose*, not only on the joint speeds.



Refresher: partial derivatives and matrix–vector products

Refresher: partial derivative

$\frac{\partial x}{\partial \theta_1}$: how much x changes when *only* θ_1 changes (all other variables frozen).

$$\dot{x} = \frac{\partial x}{\partial \theta_1}\dot{\theta}_1 + \frac{\partial x}{\partial \theta_2}\dot{\theta}_2$$

(chain rule)

Refresher: matrix times vector

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} u \\ w \end{bmatrix} = \begin{bmatrix} au + bw \\ cu + dw \end{bmatrix}$$

Each row: "this much of u plus this much of w".

Stacking the partial derivatives into a matrix gives the **Jacobian**.

The Jacobian of the 2-link arm

- Differentiate the FK equations with respect to each joint angle

$$\underbrace{\begin{bmatrix}\dot{x}\\ \dot{y}\end{bmatrix}}_{\boldsymbol{v}} = \underbrace{\begin{bmatrix}-L_1 s_1 - L_2 s_{12} & -L_2 s_{12}\\ L_1 c_1 + L_2 c_{12} & L_2 c_{12}\end{bmatrix}}_{J(\boldsymbol{\theta})} \underbrace{\begin{bmatrix}\dot{\theta}_1\\ \dot{\theta}_2\end{bmatrix}}_{\dot{\boldsymbol{\theta}}}$$

- Column 1: tip velocity per rad/s of joint 1; column 2: of joint 2
- J depends on the angles $\rightarrow$ it changes as the robot moves

The Jacobian

$$\boldsymbol{v} = J(\boldsymbol{\theta})\,\dot{\boldsymbol{\theta}}$$

Units: m/s per rad/s, i.e. metres (a lever arm).

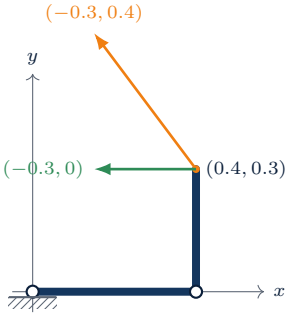
For a 6-axis arm J is 6×6: 3 linear + 3 angular velocities.

Worked example: tip velocity

Pose $\theta_1 = 0$, $\theta_2 = 90^\circ$ (tip at $(0.4,\ 0.3)$ m).

1. $s_1 = 0$, $c_1 = 1$, $s_{12} = 1$, $c_{12} = 0$
2. $J = \begin{bmatrix} -0.3 & -0.3 \\ 0.4 & 0 \end{bmatrix}$ m
3. Shoulder only, $\dot\theta = (1,\ 0)$ rad/s:
   $\boldsymbol{v} = (-0.3,\ 0.4)$ m/s, $|\boldsymbol{v}| = 0.5$ m/s
4. Elbow only, $\dot\theta = (0,\ 1)$ rad/s:
   $\boldsymbol{v} = (-0.3,\ 0)$ m/s, $|\boldsymbol{v}| = 0.3$ m/s

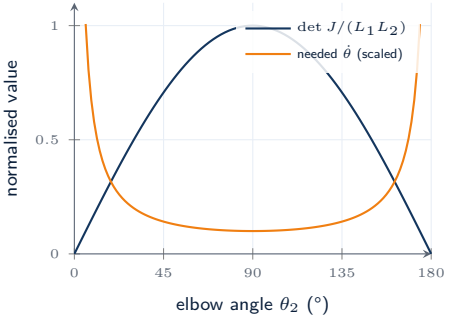
Step 3: $|\boldsymbol{v}| = \omega \cdot r$ with $r = 0.5$ m (base-to-tip distance). The door rule again.



Singularities: when the Jacobian loses a direction

- Determinant: $\det J = L_1 L_2 \sin\theta_2$
- $\det J = 0$ at $\theta_2 = 0^\circ$ (stretched) or 180° (folded): a **singularity**
- There the tip **cannot move** along the arm (radially), whatever the joints do
- Near it, a small tip speed needs **huge** joint speeds: $\dot{\boldsymbol{\theta}} = J^{-1}\boldsymbol{v}$

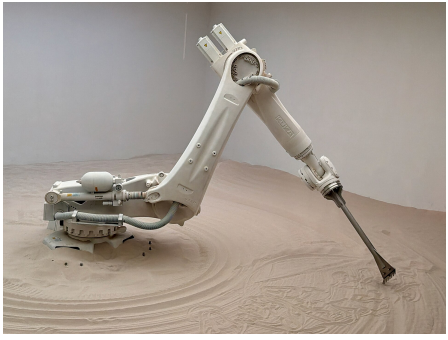
Analogy: with your arm fully stretched you cannot push your hand any further out.



Singularities of real 6-axis robots

- **Wrist singularity**: axes 4 and 6 line up (axis 5 at 0°) – two motors fight for the same rotation
- **Elbow singularity**: arm fully stretched (like our 2-link)
- **Shoulder singularity**: wrist centre directly above axis 1
- Effect: in a straight-line move the joints suddenly spin fast → the controller stops with an error
- Fix: plan paths around them, or use a joint (PTP) move

Going further: Engineers measure "distance from singularity" with the *manipulability* $\sqrt{\det(JJ^{\mathsf{T}})}$.



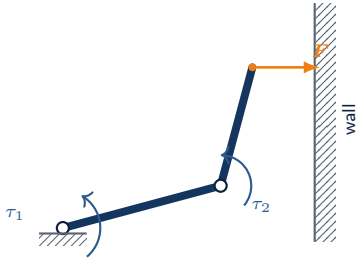
KUKA industrial robot. Oleg Yunakov, CC BY-SA 4.0, via Wikimedia Commons

Statics: from tip force to joint torques

- The robot holds a load or presses on a surface: what torque must each motor give?
- Power in $=$ power out (no motion losses): $\boldsymbol{\tau}^{\mathsf{T}}\dot{\boldsymbol{\theta}} = \boldsymbol{F}^{\mathsf{T}}\boldsymbol{v}$
- Insert $\boldsymbol{v} = J\dot{\boldsymbol{\theta}}$ and the same Jacobian appears, **transposed**:

$$\boldsymbol{\tau} = J^{\mathsf{T}}(\boldsymbol{\theta})\,\boldsymbol{F}$$

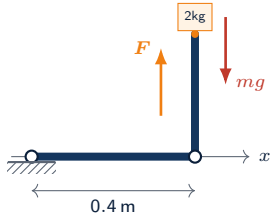
- $\boldsymbol{F}$ $=$ force the tip exerts [N], $\boldsymbol{\tau}$ $=$ joint torques [N m]



Worked example: holding a 2 kg part

Same pose as before: $\theta_1 = 0$, $\theta_2 = 90^\circ$. Links assumed massless.

1. Weight $\approx 2 \times 10 = 20$ N down, so the tip must push *up*: $\boldsymbol{F} = (0,\ 20)$ N
2. $J^{\mathsf{T}} = \begin{bmatrix} -0.3 & 0.4 \\ -0.3 & 0 \end{bmatrix}$ m
3. $\tau_1 = -0.3 \cdot 0 + 0.4 \cdot 20 = 8$ N m
4. $\tau_2 = -0.3 \cdot 0 + 0 \cdot 20 = 0$ N m



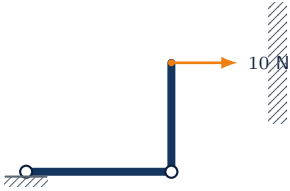
Does it make sense?

Joint 1: lever arm 0.4 m $\times$ 20 N $=$ 8 N m ✓. Joint 2: link 2 is vertical, the force passes through the joint, no lever arm ✓.

Check your understanding (4.5)

Check your understanding

1. In the pose $\theta_1 = 0$, $\theta_2 = 90^\circ$ the tip pushes on a wall with $\boldsymbol{F} = (10,\, 0)$ N. Compute τ_1 and τ_2.
2. Why does a robot moving in a straight line through a stretched-arm pose stop with an error?
3. At a singularity, is it easy or hard for the robot to *resist* a force along the arm? Think of standing with straight knees.



Session 4.5 summary and key terms

- Jacobian $J(\boldsymbol{\theta})$: joint speeds $\rightarrow$ tip velocity, $\boldsymbol{v} = J\dot{\boldsymbol{\theta}}$
- J changes with the pose
- Singularity: $\det J = 0$, a tip direction is lost, joint speeds explode nearby
- Same matrix for forces: $\boldsymbol{\tau} = J^{\mathsf{T}}\boldsymbol{F}$
- Used for motor sizing, force control and numerical IK

Key terms

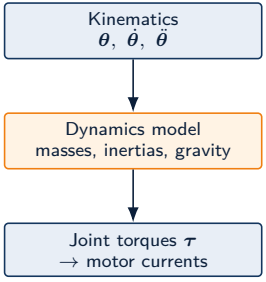
Jacobian, **velocity kinematics**, **determinant**, **singularity** (wrist / elbow / shoulder), **manipulability**, **statics**, **virtual work**

Session 3

4.6 Introduction to robot dynamics

Why dynamics? Geometry is not enough

- Kinematics: *where* and *how fast* (geometry only)
- Dynamics: *which torques* are needed to make that motion happen
- Analogy: carrying a full bucket with your arm stretched out vs. close to your body – same bucket, very different effort
- Needed for: motor sizing (W1D2), feedforward and computed-torque control (W1D4), simulation



Refresher: rotation, energy and the Lagrange idea

Refresher: Newton for rotation

$\tau = J\alpha$: torque [N m] = inertia [kg m^2] $\times$ angular acceleration [rad/s^2]. Point mass at radius r: $J = mr^2$.

Refresher: energy

Kinetic $K = \frac{1}{2}mv^2$, potential $P = mgh$.

Lagrange in one sentence

Write the total kinetic energy K and potential energy P of the arm; a standard recipe (differentiate $K - P$) gives the torque each joint needs.

Going further: The recipe is $\tau_i = \frac{d}{dt}\frac{\partial L}{\partial \dot{\theta}_i} - \frac{\partial L}{\partial \theta_i}$ with $L = K - P$. Robot software uses the faster Newton–Euler recursion.

The robot equation of motion

$$\boldsymbol{\tau} = \underbrace{M(\boldsymbol{\theta})\,\ddot{\boldsymbol{\theta}}}_{\text{inertia}} + \underbrace{C(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})\,\dot{\boldsymbol{\theta}}}_{\text{Coriolis \& centrifugal}} + \underbrace{\boldsymbol{g}(\boldsymbol{\theta})}_{\text{gravity}} \; (+ \text{ friction})$$

Inertia term

M: like J in $\tau = J\alpha$, but a matrix that **changes with the pose**.

Velocity term

Forces that appear because links rotate; grow with speed squared. Small at low speed.

Gravity term

Torque just to **hold** the pose. Present even at standstill.

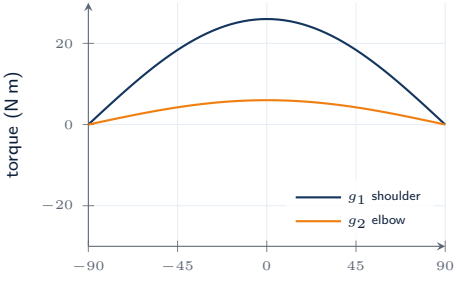
Same structure as a mass–spring–damper (W1D2): "mass $\times$ acceleration $+$ velocity term $+$ position term $=$ force".

Gravity torque of the 2-link arm

- Simple model: point masses m_1 at the elbow, m_2 at the tip (link 2 + payload)
- Torque = weight $\times$ horizontal lever arm

$$g_2 = m_2\, g\, L_2 \cos(\theta_1 + \theta_2)$$
$$g_1 = (m_1 + m_2)\, g\, L_1 \cos\theta_1 + g_2$$

- Maximum when the arm is **horizontal**, zero when vertical

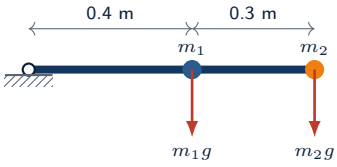


$m_1 = 3\,\text{kg}$, $m_2 = 2\,\text{kg}$, $g \approx 10\,\text{m/s}^2$

Worked example: holding the arm horizontal

$m_1 = 3\,\text{kg}$, $m_2 = 2\,\text{kg}$, $L_1 = 0.4\,\text{m}$, $L_2 = 0.3\,\text{m}$, $\theta_1 = \theta_2 = 0$.

1. Elbow: $g_2 = 2 \cdot 10 \cdot 0.3 = 6\,\text{N m}$
2. Shoulder, own part: $(3 + 2) \cdot 10 \cdot 0.4 = 20\,\text{N m}$
3. Shoulder total: $g_1 = 20 + 6 = 26\,\text{N m}$
4. Add a 1 kg payload ($m_2 = 3\,\text{kg}$): $g_2 = 9\,\text{N m}$, $g_1 = 24 + 9 = 33\,\text{N m}$



Take-away

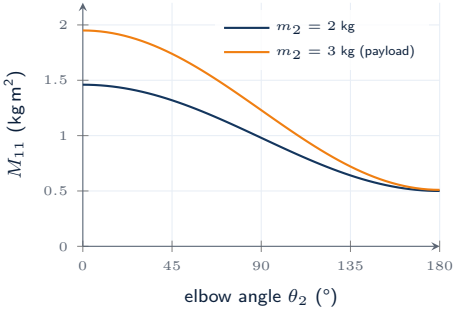
1 kg more payload costs 7 N m at the shoulder: the lever arm is the whole reach (0.7 m).

Inertia changes with the pose

- Inertia seen by the shoulder (point masses):

$$M_{11} = m_1 L_1^2 + m_2\left(L_1^2 + L_2^2 + 2L_1 L_2 \cos\theta_2\right)$$

- Stretched ($\theta_2 = 0$):
  $0.48 + 2 \cdot 0.49 = 1.46\,\text{kg m}^2$
- Folded ($\theta_2 = 180°$):
  $0.48 + 2 \cdot 0.01 = 0.50\,\text{kg m}^2$
- Factor $\approx 3 \rightarrow$ a fixed PID tuning behaves differently in different poses



Analogy: a figure skater pulls in the arms and spins faster.

Payload effects and Coriolis forces

Payload

- More payload $\rightarrow$ more gravity torque *and* more inertia
- Datasheet payload (e.g. 10 kg) is valid only at a given distance from the flange
- Robot controllers ask for payload mass and centre of gravity: wrong data $\rightarrow$ overshoot, path errors, false collision alarms

Coriolis and centrifugal

- Analogy: walking outward on a turning merry-go-round, you are pushed sideways
- Grow with $\dot{\theta}^2$: unimportant when slow, large for fast pick-and-place

Going further: *Computed-torque control* (W1D4) uses M, C, $\boldsymbol{g}$ to cancel the robot's own dynamics; delta robots reach $> 100\,\mathrm{m/s^2}$ partly because their moving mass is so small.

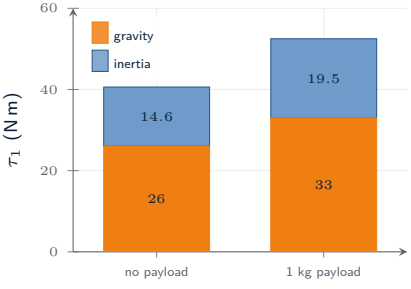
Worked example: torque to accelerate the shoulder

Arm stretched and horizontal, start moving up with $\ddot{\theta}_1 = 10\,\text{rad/s}^2$ (from rest, so no Coriolis term).

1. $\tau_1 = M_{11}\,\ddot{\theta}_1 + g_1$
2. No payload: $1.46 \cdot 10 + 26 = 14.6 + 26 = 40.6\,\text{N m}$
3. 1 kg payload: $M_{11} = 0.48 + 3 \cdot 0.49 = 1.95\,\text{kg m}^2$
   $\tau_1 = 19.5 + 33 = 52.5\,\text{N m}$

Take-away

Gravity is the larger part here; payload adds about 30 % torque.



Check your understanding (4.6)

Check your understanding

1. The arm points straight up ($\theta_1 = 90°$, $\theta_2 = 0$). What are the gravity torques g_1 and g_2?
2. Why does a robot axis need more torque to accelerate when the arm is stretched than when it is folded?
3. Which term of the equation of motion can you ignore for a slow painting robot, and which never?

Session 4.6 summary and key terms

- $\boldsymbol{\tau} = M(\boldsymbol{\theta})\ddot{\boldsymbol{\theta}} + C\dot{\boldsymbol{\theta}} + \boldsymbol{g}(\boldsymbol{\theta})$
- Gravity: holding torque, largest with the arm horizontal
- Inertia depends on the pose (factor 3 in our arm)
- Payload raises both gravity and inertia torques
- Coriolis/centrifugal matter only at high speed

Key terms

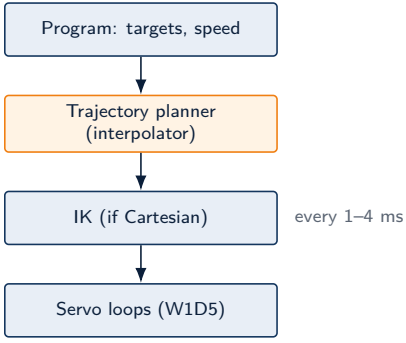
dynamics, **Lagrangian**, **inertia (mass) matrix**, **gravity torque**, **Coriolis / centrifugal**, **payload**, **computed torque**

Session 4

4.7 Motion systems and trajectory planning

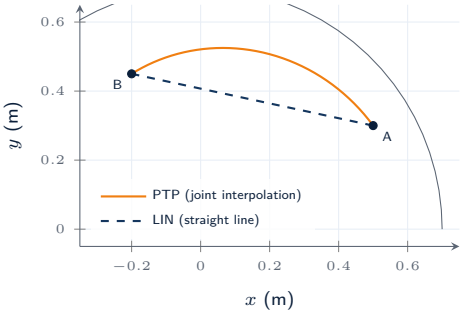
Path and trajectory

- **Path**: the geometric curve the tool follows (*where*)
- **Trajectory**: path + timing: position, velocity, acceleration at each instant (*where and when*)
- Analogy: a route on a map vs. a train timetable
- The planner sends new set-points to every axis each **interpolation cycle** (typically 1–4 ms)



Point-to-point (PTP) moves in joint space

- Interpolate each **joint angle** from start to end
- All joints start and stop together (**synchronised PTP**)
- Fastest move, no IK along the way, no singularity trouble
- But the tool path is a **curve**, not easy to predict
- Used for free moves through the air



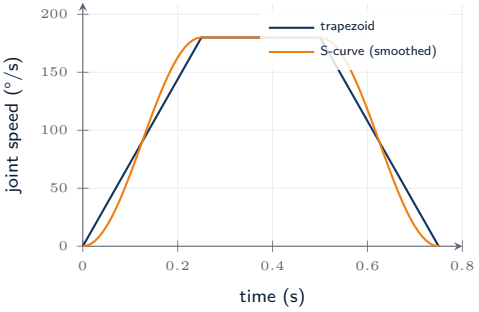
Velocity profiles: trapezoid and S-curve

- Recap W1D5: accelerate, cruise, decelerate
- **Trapezoidal** profile: limited velocity and acceleration
- **S-curve**: also limits **jerk** (rate of change of acceleration) → less shaking

Worked example

$\Delta\theta = 90°$, $\omega_{\max} = 180\,°/\text{s}$, $\alpha = 720\,°/\text{s}^2$

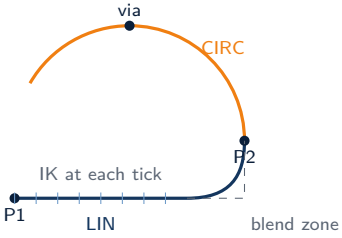
1. $t_a = 180/720 = 0.25\,\text{s}$, angle $\frac{1}{2} \cdot 720 \cdot 0.25^2 = 22.5°$
2. Accel + decel: 45°; cruise 45° in 0.25 s
3. Total: $0.25 + 0.25 + 0.25 = 0.75\,\text{s}$



The S-curve here (sine-shaped acceleration) has the same duration but peak acceleration $\approx 1.6\times$ higher.

Linear (LIN) and circular (CIRC) moves in Cartesian space

- **LIN**: tool centre point (TCP) moves on a straight line with constant orientation change
- **CIRC**: arc through a **via point** to the end point
- Planner cuts the path into small steps and runs **IK at every step**
- Needed for welding, gluing, insertion, cutting
- Risk: passing near a singularity → joint speed limit error
- **Blending** (fly-by): round the corners between moves to avoid stopping



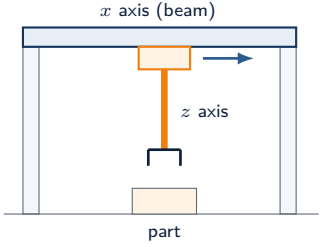
Joint space vs. Cartesian space: when to use which

	Joint space (PTP)	Cartesian space (LIN / CIRC)
Tool path	curved, depends on robot	straight line or arc, predictable
Speed	fastest (each joint near its limit)	limited by the slowest joint along the path
Computation	interpolate angles only	IK every cycle
Singularities	no problem	can stop the move
Configuration	may change (elbow flips)	must stay the same
Typical use	approach, free transfer, retract	process path: weld, glue, insert

Rule of thumb: **PTP through the air, LIN near the part.** A typical pick: PTP to above the part, LIN down, grip, LIN up, PTP away.

Linear axes and gantries: the easy kinematics

- **Cartesian / gantry robot**: three prismatic axes along x, y, z
- FK and IK are trivial: $x = q_1$, $y = q_2$, $z = q_3$; no singularities
- Drives: ball screw, belt, rack-and-pinion or linear motor (W1D2)
- Large, stiff, accurate: CNC loading, 3D printers, palletising over long distances
- Often a **7th axis**: a linear track carrying an articulated robot

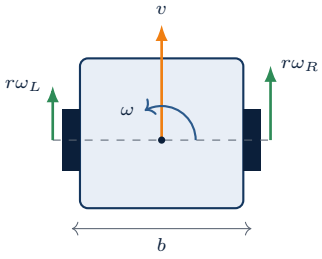


Differential-drive mobile robots (AMRs)

- Two driven wheels (radius r) on one axle, track width b, plus caster wheels
- Same wheel speeds: straight; different: turn; opposite: spin on the spot

$$v = \frac{r(\omega_R + \omega_L)}{2}, \qquad \omega = \frac{r(\omega_R - \omega_L)}{b}$$

- Cannot move sideways (**nonholonomic**) – like a shopping trolley with fixed rear wheels



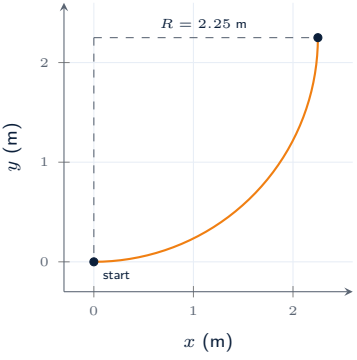
Worked example: an AMR turning in a warehouse

$r = 0.1\,\text{m}$, $b = 0.5\,\text{m}$, $\omega_R = 10\,\text{rad/s}$, $\omega_L = 8\,\text{rad/s}$.

1. $v = 0.1 \cdot (10 + 8)/2 = 0.9\,\text{m/s}$
2. $\omega = 0.1 \cdot (10 - 8)/0.5 = 0.4\,\text{rad/s}$
3. Turning radius $R = v/\omega = 2.25\,\text{m}$
4. Quarter circle takes $(\pi/2)/0.4 \approx 3.9\,\text{s}$



AGV pallet stacker. Marc Auledas Noguera, CC BY 4.0, via Wikimedia Commons



Check your understanding (4.7)

Check your understanding

1. A robot must apply glue along the straight edge of a car window. PTP or LIN? Why?
2. Our arm moves from A to B with a PTP move. Why does the tool not travel in a straight line?
3. An AMR has $\omega_R = 5\,\text{rad/s}$ and $\omega_L = -5\,\text{rad/s}$ ($r = 0.1\,\text{m}$, $b = 0.5\,\text{m}$). Describe its motion.

Session 4.7 summary and key terms

- Trajectory = path + timing; planner runs every few ms
- PTP: interpolate joints, fast, curved path
- LIN/CIRC: interpolate the TCP, IK each cycle, watch singularities
- Gantries: trivial kinematics, large and stiff
- Differential drive: v from the wheel average, ω from the difference

Key terms

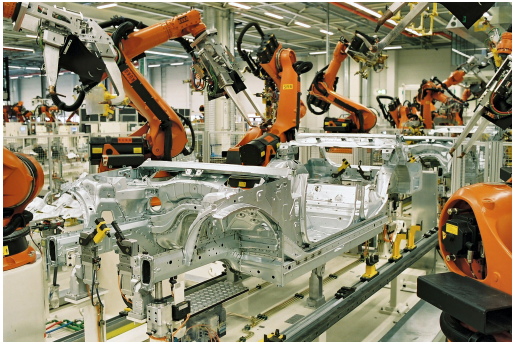
path, **trajectory**, **PTP**, **LIN / CIRC**, **TCP**, **blending**, **jerk**, **gantry**, **differential drive**, **nonholonomic**

Session 5

5.1 Robotic manufacturing and flexible automation

Why automate?

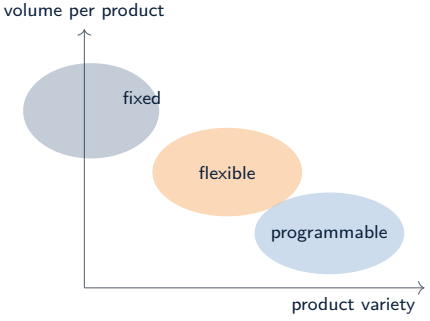
- **Quality**: same weld, same glue bead, every time
- **Cost**: lower cost per part at high volume, 24/7 operation
- **Safety**: the “3 D” jobs – dull, dirty, dangerous
- **Speed**: shorter cycle times, faster delivery
- Limits: high investment, needs skilled people, inflexible if badly planned



Spot welding of car bodies, BMW plant Leipzig. BMW Werk Leipzig, CC BY-SA 2.0 de, via Wikimedia Commons

Fixed, programmable and flexible automation

- **Fixed (hard)**: special machine, one product, very high volume (bottle filling, transfer lines)
- **Programmable**: re-programmed between *batches*; changeover takes time (CNC, robots with new programs)
- **Flexible**: switches between known product variants **without lost time** (FMS, robot cells with tool changers)



Comparing the three types

	Fixed	Programmable	Flexible
Volume	very high	low to medium (batches)	medium
Variety	one product	many, one batch at a time	a family, mixed
Changeover	rebuild the machine	hours (new program, tooling)	almost none
Investment	high, special purpose	medium, general purpose	highest (machines + control)
Example	bottling line	CNC lathe, robot with new program	FMS, multi-model robot cell

Industrial robots are *programmable* machines; arranged with tool changers, vision and a cell controller they become the core of *flexible* automation.

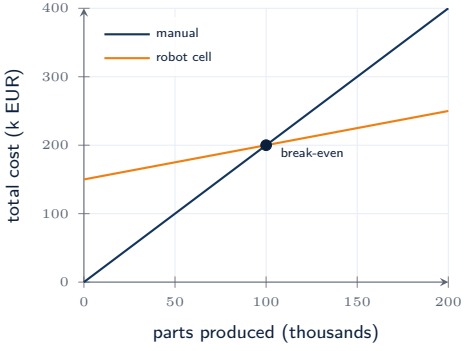
Worked example: when does a robot cell pay off?

Cost model: total = fixed cost + cost per part $\times N$

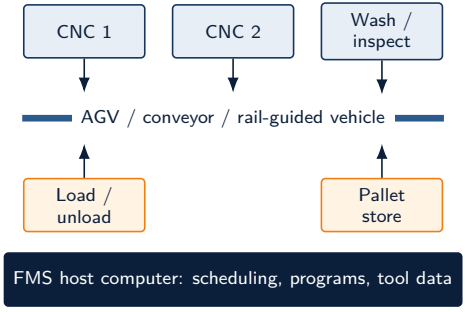
1. Manual: EUR 0 + EUR 2.00 per part
2. Robot cell: EUR 150 000 + EUR 0.50 per part
3. Break-even: $2N = 150\,000 + 0.5N$ $\Rightarrow N = 150\,000/1.5 = 100\,000$ parts
4. At 400 parts/day $\rightarrow$ 250 days $\approx$ 1 year

Take-away

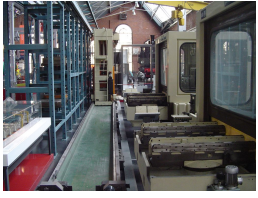
Automation pays off with **volume**. Flexibility lets one cell reach that volume with several products.



Flexible manufacturing systems (FMS)

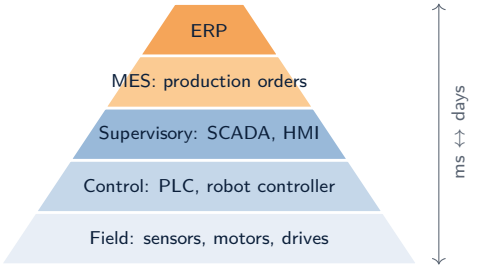


- Several CNC machines or robot cells
- Automated material handling between them
- Central computer routes each part
- Runs mixed parts, often unattended at night



FMS with rail-guided pallet transport. Kolossos, CC BY-SA 3.0, via Wikimedia Commons

Where does the robot sit? The automation pyramid

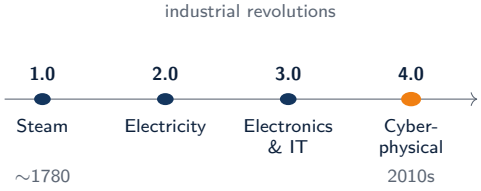


- Lower levels: fast (ms), small data, real-time
- Upper levels: slow (hours, days), business data
- Robots and PLCs at the **control** level (W2D3)
- Industrial networks connect the levels (W2D5)

Going further: The levels follow the ISA-95 / IEC 62264 model.

Industry 4.0: the connected factory

- **Cyber-physical system**: machine + computer + network
- **Industrial IoT**: sensors on everything, data to the cloud/edge
- **Digital twin**: a live simulation model of a cell (W2D3)
- Uses: predictive maintenance, batch size one, traceability
- Collaborative robots and AMRs make cells easy to re-arrange



Measuring a production cell: cycle time and OEE

- **Cycle time**: time between two finished parts
- **Takt time**: available time / demand – the cycle time the customer needs
- **OEE** (overall equipment effectiveness):

$$\text{OEE} = \text{Availability} \times \text{Performance} \times \text{Quality}$$

Worked example

1. Shift 8 h, demand 960 parts: takt $= 28\,800/960 = 30$ s
2. Cell down 48 min: A $= 432/480 = 0.90$
3. Runs at 95 % of ideal speed: P $= 0.95$
4. 1 % scrap: Q $= 0.99$
5. OEE $= 0.90 \cdot 0.95 \cdot 0.99 \approx 0.85$

Check your understanding (5.1)

Check your understanding

1. A bakery makes one kind of bread roll, 50 000 per day, for years. Which type of automation fits? And a job shop making 20 different brackets in batches of 50?
2. What makes a robot cell *flexible* rather than only *programmable*?
3. In the break-even example, the robot cell costs EUR 0.50 per part. What happens to the break-even volume if a cheaper gripper brings it to EUR 0.25?

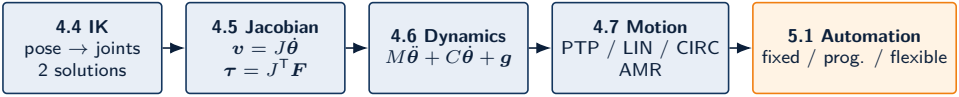
Session 5.1 summary and key terms

- Automate for quality, cost, safety and speed
- Fixed $\rightarrow$ programmable $\rightarrow$ flexible: more variety, less volume per product
- Break-even: fixed cost / saving per part
- FMS = machines + automated handling + central computer
- Industry 4.0: connected, data-driven, re-configurable

Key terms

fixed / programmable / flexible automation, **changeover**, **FMS**, **automation pyramid**, **Industry 4.0**, **digital twin**, **cycle time**, **takt time**, **OEE**

Day summary



Three things to remember

1. IK can have several answers – or none
2. One matrix J links speeds *and* forces; avoid $\det J = 0$
3. Gravity and payload dominate the torque of a slow arm

From robot to factory

The math of one arm (kinematics, dynamics, trajectories) is what makes a robot *programmable*; putting robots, handling and computers together makes production *flexible*.

Preview of tomorrow (W2D3): manufacturing and automation

- **5.2** Industrial robot applications: welding, painting, palletising, machine tending, assembly, pick-and-place
- **5.3** Work-cell design: layout, reach, cycle time, end-of-arm tooling
- **5.4** PLCs: scan cycle, I/O, ladder logic, IEC 61131-3
- **5.5** Robot programming: teach pendant, base/tool/user frames, offline programming, digital twin



Delta X robot. Trungdoanhong, CC BY-SA 4.0, via Wikimedia Commons

Today's reachability check returns tomorrow as cell layout.

Figure credits

All photographs from Wikimedia Commons; all other drawings and plots are original (TikZ/pgfplots).

- SCARA robot (4.4): “SCARA robot 2R.png”, Mitsubishi Electric Automation, Inc., CC BY-SA 4.0
- KUKA industrial robot (4.5): “KUKA industrial robotic arm performing ... - 02.jpg”, Oleg Yunakov, CC BY-SA 4.0
- Mobile robot (4.7): “Jungheinrich ERC 213a 217a Automated Guided Vehicle.jpg”, Marc Auledas Noguera (Auledas), CC BY 4.0
- Robot welding (5.1): “BMW Leipzig MEDIA 050719 Download Karosseriebau max.jpg”, BMW Werk Leipzig, CC BY-SA 2.0 de
- Flexible manufacturing system (5.1): “FlexiblesFertigungssystem.jpg”, Kolossos, CC BY-SA 3.0
- Delta robot (preview): “Delta x robot.jpg”, Trungdoanhong, CC BY-SA 4.0

Full URLs: `figures/CREDITS.tsv` (lines with prefixes `r22_` and `d4_kuka`).

References

- J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed., Pearson, 2018.
- M. W. Spong, S. Hutchinson, M. Vidyasagar, *Robot Modeling and Control*, 2nd ed., Wiley, 2020.
- K. M. Lynch, F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Cambridge University Press, 2017.
- B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2009.
- P. Corke, *Robotics, Vision and Control*, 3rd ed., Springer, 2023.
- R. Siegwart, I. R. Nourbakhsh, D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed., MIT Press, 2011.
- M. P. Groover, *Automation, Production Systems, and Computer-Integrated Manufacturing*, 5th ed., Pearson, 2019.
- ISO 9787:2013, Robots and robotic devices – Coordinate systems and motion nomenclatures.
- IEC 62264-1:2013, Enterprise-control system integration – Part 1: Models and terminology.